

# **VitYarthi Project :**

## **Hospital Triage & Appointment Scheduling Simulation**

**A Console-Based Clinical Prioritization & Real-Time Slot  
Reservation Platform.**

**Submitted By – Gaurav Rathi**

**Registration Number – 26BCE11407**



**SCHOOL OF COMPUTER SCIENCE AND ENGINEERING**

**VIT BHOPAL UNIVERSITY**

**KOTRIKALAN, SEHORE**

**MADHYA PRADESH – 466114**

**September , 2026**

# **INTRODUCTION**

Modern healthcare facilities frequently operate under intense operational pressure, where timely clinical prioritization and structured outpatient administration directly influence patient outcomes. When emergency rooms and outpatient departments experience surges in patient volume, relying on manual, subjective assessments creates delays in recognizing deteriorating patients. Furthermore, disconnected communication between reception desks and specialist rosters often causes appointment overlaps, double-booking, and inefficient allocation of medical personnel.

Hospital Triage Simulation & Doctor Appointment Scheduling System:

It is designed as a centralized, interactive console application that integrates emergency intake triage with specialist consultation scheduling. Developed using standard Python data structures and algorithms, the system addresses these operational bottlenecks through two core mechanisms

## **1. Automated Triage Risk Assessment:**

Upon intake, the application records vital physiological indicators—specifically blood oxygen saturation ( $SpO_2$ ), heart rate, and patient age. Using a rule-based clinical scoring engine, it quantifies physiological risk and classifies patients into priority tiers (NORMAL, URGENT!, or CRITICAL!), providing objective visibility over high-risk cases.

## **2. Dynamic Specialist Scheduling Engine:**

The system manages appointment rosters across four specialized departments: Cardiology, Orthopaedics, Oncology, and Pulmonology. Users can schedule appointments through dual booking pathways—either by doctor preference or target consultation date. To prevent scheduling conflicts, booked slots are depleted in real time from the physician's schedule. A provider-facing interface enables doctors to monitor triage records, review booked consultations, and track remaining open slots.

By merging vital-sign triage prioritization with conflict-free slot management, the application demonstrates how core computational techniques can streamline medical workflows and improve administrative reliability in healthcare environments.

# **Problem Statement**

Problem Statement & System Requirements Specification

Platform : Python 3.14

## **1.) Problem Statement:**

In both outpatient departments and emergency healthcare intake units, two critical administrative and clinical bottlenecks consistently emerge:

Subjective Emergency Triage & Delayed Risk Identification:

Manual, unstructured patient intake relies heavily on subjective visual judgment or simple first-come-first-served queues. In high-pressure environments, this results in human error where patients suffering from silent, acute physiological distress (such as severe hypoxemia or abnormal tachycardia) wait behind stable patients, substantially elevating the risk of preventable morbidity and mortality.

## **2.) Scheduling Conflicts and Inflexible Consultation Booking:**

Coordinating outpatient consultations across multiple clinical specialties (Cardiology, Orthopaedics, Oncology, and Pulmonology) using decentralized or manual registries frequently causes:

Double-booking:

Multiple patients receiving the same specialist slot.

Rigid Scheduling Pathways: Systems typically force users into a single booking flow, failing to support patients who prioritize a specific physician versus those who need a consultation on a specific date.

Provider-Patient Desynchronization: Physicians lack real-time visibility into their booked consultations, remaining availability, and the incoming emergency triage queue.

The objective of this project is to implement a unified, algorithmic decision-support and scheduling simulation in Python that automates patient urgency triage using physiological markers and provides dynamic, conflict-free appointment management.

### **3.) Scope of the Project In-Scope:**

Algorithmic Triage Prioritization: Quantitative risk scoring based on physiological vital inputs: Oxygen Saturation (SpO2), Heart Rate (BPM), and Age.

Urgency Stratification: Automated classification of patient intake into three distinct priority levels: NORMAL, URGENT!, and CRITICAL!.

Specialty & Physician Roster Management: In-memory modeling of multiple medical disciplines (Cardiologist, Orthopedic, Oncologist, Pulmonologist) with distinct medical staff and date rosters.

Bi-Directional Booking Engine: Dual booking pipelines allowing users to book either by specialist preference (Doctor-First) or by calendar schedule availability (Date-First).

Dynamic Slot Depletion (Collision Prevention): Real-time removal (pop/remove) of consultation dates from a doctor's schedule upon successful booking to guarantee zero duplicate reservations.

Provider Dashboard: A dedicated physician portal enabling doctors to review hospital triage patients, inspect patient appointments booked under their name, and audit remaining available dates.

Input Validation: Numeric constraints and type-checking on vitals, menu options, and roster indices to prevent runtime exceptions.

Out-of-Scope:

Persistent disk or external relational database integration (all state management is handled in-memory during runtime).

Graphical User Interface (GUI) or web application frontend (interaction is strictly terminal/CLI-based).

External SMS, email, or webhook notification systems for appointment confirmation.

Integration with clinical hardware/IoT sensors for direct vitals capture.

Patient billing, health insurance processing, and electronic health record (EHR) export.

### **4. ) Target Users:**

The system is designed to simulate workflows utilized by three primary user groups:

Triage Nurses & Emergency Reception Staff:

Front-desk healthcare personnel who record arriving patient vitals, verify physiological parameters, and require instantaneous, algorithmic priority classification to direct critical patients to immediate emergency care.

Outpatient Coordinators & Helpdesk Staff / Patients:

Users booking specialist consultations who require flexible filtering—either matching a patient with a specific expert doctor or finding any available specialist on an urgent, specific calendar date without scheduling collisions.

Consulting Physicians & Medical Specialists:

Hospital doctors who need direct visibility into their scheduled consultations, accurate audits of their open calendar dates, and cross-departmental awareness of pending emergency triage cases.

## **5.) High-Level-Features:**

### **A.) Quantitative Vital-Signs Triage Engine**

Captures physiological vitals: SpO2, Heart Rate, and Age.

Evaluates clinical risk using a multi-factor tiered weighting formula:

Oxygen Saturation:

(SpO2): <90% then (+5 points),

90% - 93% then (+3 points).

Heart Rate (BPM):

130 or <40(BPM) then (+4 points),

111 - 130 or 40 - 49 (BPM) then (+2 points).

Age Demographics: >60(years) then (+1 point).

Maps scores to immediate action classifications:

(Score)>=6 then CRITICAL!

Score(3 - 5) then URGENT!

(Score) < 3 then NORMAL

Appends comprehensive intake records into a centralized patient registry.

### **B.) Bi-Directional Specialist Booking System:**

Pathway 1 (Priority by Doctor):

Specialty = Doctor the Available Date is Confirmation.

Pathway 2 (Priority by Date):

Specialty = Target Hospital Date then 'List of Available Specialists' on that Day then 'Confirmation'.

Autonomous Calendar Depletion:

Immediately isolates and removes confirmed calendar dates from the target doctor's list, ensuring complete concurrency safety within the session.

### **C.) Physician Management Interface**

Triage Queue Inspection:

Comprehensive audit of all triage-screened patients and their priority ratings.

Doctor-Specific Appointment Search:

Case-insensitive query system allowing doctors to review their upcoming consultations along with patient names and booked dates.

Remaining Slot Auditing:

Real-time visibility into still-available open consultation dates for any physician in the directory.

## **D.) System Diagnostics & Error Handling**

Safe input sanitization utilizing `.strip()`, `.lower()`, and `.isdigit()` checks.

Guard rails preventing invalid menu entries, out-of-range date selections, and negative vital-sign inputs.

This file covers:

Problem Statement: Detailed breakdown of manual triage subjectivity and scheduling collision bottlenecks.

Scope of the Project: Clear distinction of both in-scope features and out-of-scope boundaries.

- (i) Target Users: Personas for triage staff, booking coordinators/patients, and medical specialists.
- (ii) High-Level Features: Clinical triage scoring logic, bi-directional booking mechanics, dynamic calendar depletion, and the doctor interface.
- (iii) Visual Workflow Diagram: Demonstrates the end-to-end operational flow from triage to schedule mutation and doctor audit

## **Functional Requirements**

### **1.) Patient Registration and Vital Sign Triage**

(i) Data Input Collection:

Captures demographic and physiological values: patient name, age, oxygen saturation level ( $SpO_2$ ), and heart rate.

(ii) Input Validation:

Checks whether numeric values for age, oxygen, and heart rate are non-negative.

(iii) Algorithmic Risk Scoring:

Calculates a weighted risk score based on vital sign thresholds:

Oxygen levels: +5 points if  $< 90\%$ , +3 points if  $< 94\%$ .

Heart rate: +4 points if  $> 130$  or  $< 40$  bpm, +2 points if  $> 110$  or  $< 50$  bpm.

Age: +1 point if  $> 60$  years

(iv) Priority Categorization:

Maps the cumulative score into distinct urgency tiers:

- Score  $\geq 6 \rightarrow$  CRITICAL!
- Score  $\geq 3 \rightarrow$  URGENT!

- Score < 3 → NORMAL

(v) Record Storage: Appends the patient's vitals, score, and priority tier as a structured tuple.

## **2.) Triage Queue Display:**

(i) Queue Inspection:

Checks whether the patient registry contains records.

(ii) Queue Formatting:

Displays all registered triage patients and their recorded parameters, or returns an alert if no patients are currently logged.

## **3.) Doctor Portal Interface:**

(i) Triage Data Access:

Allows physicians to view the complete list of queued triage patients from within the portal.

(ii) Personalized Appointment Filtering:

Queries and displays all booked appointments matching a specific doctor's name using case-insensitive search.

(iii) Availability Audit:

Searches the directory and displays all remaining, unbooked consultation dates for a specified doctor, or alerts if the doctor is fully booked.

## **4. Specialist Appointment Booking Engine:**

(i) Specialty Directory Navigation:

Allows users to select from four medical specialties: Cardiologist, Orthopedic, Oncologist, or Pulmonologist.

(ii) Priority by Doctor:

- Displays all doctors within the selected specialty.
- Displays the selected doctor's open dates.
- Pops the chosen date from the doctor's calendar via `.pop()` to prevent double-booking.

(iii) Priority by Date:

- Displays the hospital's master date list (`all_hospital_dates`).
- Filters and lists only the doctors in the chosen specialty who have that specific date available.
- Removes the selected date from that doctor's schedule via `.remove()`.

(iv) Booking Confirmation & Logging:

Appends a dictionary containing patient name, specialty, doctor, and date into the appointments list and outputs a booking confirmation.

## **5.) Session and Loop Management:**

Navigation Control:

Keeps the system running continuously via a while True loop until Option 5 is selected, gracefully terminating execution.

## **Non-Functional Requirements**

### **1.) Reliability and Data Consistency**

- (i) When a doctor's slot is booked, the date is modified in place using `pop()` (in doctor-first booking) or `remove()` (in date-first booking).
- (ii) This prevents double-booking and guarantees that no two patients can claim the same consultation slot.
- (iii) Initializing doctor calendars with `list(all_hospital_dates)` ensures that each doctor gets an independent copy of the date list, preventing mutations from leaking across physicians.

## **2.) Input Validation and Error Handling**

- (i) The code uses `.isdigit()` and range comparisons (examples , `int(doc_input) < 1` or `int(doc_input) > len(doctor_list)`) to catch invalid choices without crashing the program.
- (ii) Boundary checks alert the user when entering negative values for physiological vitals.
- (iii) Default else conditions handle unrecognized menu selections and return the user safely to the menu loop.

## **3.) Usability and User experience**

- (i) Uses sequential, numbered command-line menus driven by `enumerate(..., 1)` to provide 1-based indexing for easy selection.
- (ii) Implements `.strip().lower()` for physician name queries so doctor searches are resilient to extra whitespace and case mismatches.
- (iii) Displays user-facing alerts and confirmation summaries immediately after triage calculations or appointment bookings.

## **4.) Performance and Resource Efficiency**

- (i) The program runs fully in memory using standard Python built-in collections (list, dict, tuple), eliminating external network, database, or disk I/O latency.
- (ii) Specialty lookups inside `doctors_directory` operate with  $O(1)$  dictionary access time.
- (iii) Operates with a minimal memory footprint and zero third-party library dependencies.

# **System Architecture**

## **1.) System Architecture**

The VITyarthi Hospital Triage Simulation is designed as a menu-driven, command-line-based Python application. The system combines two major functionalities:

- 1. Hospital Triage Management**
- 2. Doctor Appointment Management**

The system accepts information from the user, processes the information using predefined rules, stores the resulting records in memory, and displays the appropriate output.

The architecture follows a simple layered logical structure in which the user interacts with the main menu, the system processes the selected operation, and the required data structures are updated.

The overall architecture can be represented as:

**User → Main Menu → Selected Module → Processing/Decision Logic → Data Storage  
→ Output**

The system does not currently use an external database. Patient information and appointment information are stored during program execution using Python lists and dictionaries.

### **Major Components**

The system consists of the following major components:

The main menu acts as the entry point of the application.

The user is provided with five choices:

- Add patients
- Show patients
- Doctor's interface
- Book Appointment
- Exit

The program continuously displays the menu using a while True loop until the user selects the exit option.

The main menu therefore controls the overall workflow of the application.

### **Patient Registration and Triage Module**

This module collects important patient information:

- Patient name
  - Age
- Oxygen level
- Heartbeat rate

The collected information is then processed to calculate a triage score.

The program evaluates oxygen level, heartbeat rate, and age using predefined conditions.

The triage score is calculated as follows:

- Oxygen level below 90 → 5 points
- Oxygen level below 94 → 3 points
- Heartbeat above 130 or below 40 → 4 points
- Heartbeat above 110 or below 50 → 2 points
- Age above 60 → 1 point

After calculating the score, the patient is classified into one of three priority levels:

| <b>Score</b> | <b>Priority</b> |
|--------------|-----------------|
| 6 or above   | CRITICAL        |
| 3–5          | URGENT          |
| Below 3      | NORMAL          |

The resulting patient information is stored in the patient list.

### **Patient Display Module**

The patient display module allows the user to view all patients currently registered during the program execution.

If there are no patients in the system, the program displays:

**"No patients registered."**

Otherwise, the system iterates through the patient list and displays each stored patient record.

This provides a simple way to review the triage information generated by the system.

### **Doctor's Interface Module**

The Doctor's Interface provides doctors with access to relevant hospital information.

It contains three major operations:

1. View all triage patient data
2. View booked appointments
3. View remaining available dates

The doctor interface operates inside its own loop, allowing the user to perform multiple operations before returning to the main menu.

The interface retrieves information from the same patient and appointment data structures used by the main system.

### **Appointment Booking Module**

The appointment module allows a patient to book an appointment with a doctor.

The system supports two appointment-selection preferences:

1. Priority by Doctor
2. Priority by Date

For doctor-based booking, the user first selects a medical specialty, then a doctor, followed by an available date.

For date-based booking, the user first selects a specialty and hospital date. The system then displays doctors from that specialty who are available on the selected date.

Once an appointment is successfully booked, the selected date is removed from that doctor's available-date list. The appointment is then stored in the appointments list.

## Data Structures Used

The project primarily uses Python's built-in data structures.

### Patient List

```
patient = []
```

This list stores the records of registered patients.

Each patient record contains:

- Name
- Age
- Oxygen level
- Heart rate
- Triage score
- Priority

### Appointment List

```
appointments = []
```

This list stores appointment records containing:

- Patient name
- Specialty
- Doctor
- Date

### Doctor Directory

The doctors\_directory dictionary organizes doctors according to their medical specialty.

The current specialties are:

- Cardiologist
- Orthopaedic
- Oncologist
- Pulmonologist

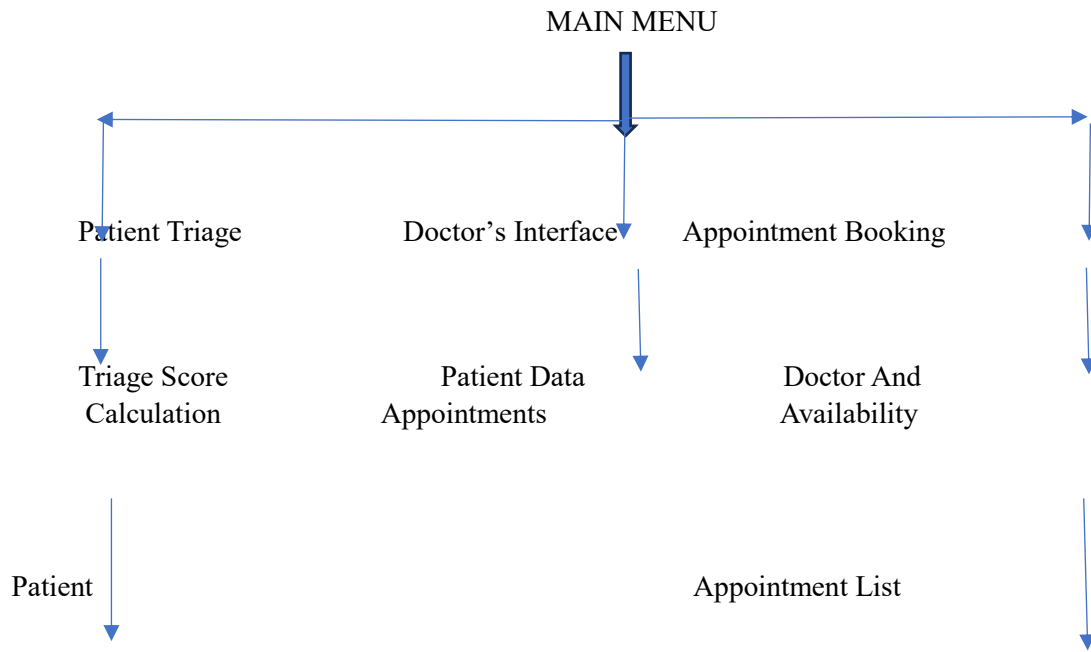
Each specialty contains multiple doctors and their available dates.

## System Architecture Diagram

The architecture of the system can be represented as follows:

USER





## Design Diagrams

The main actor in the current implementation is the **User**.

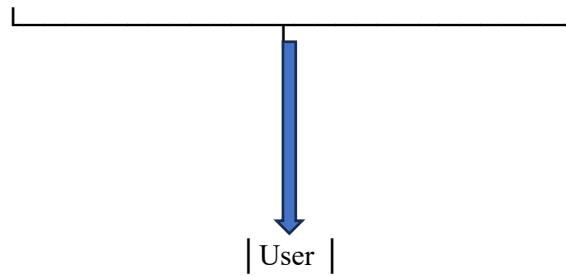
The user can perform the following operations:

- Add patient
- View patient records
- Access doctor interface
- View appointments
- View doctor availability
- Book appointment
- Exit the system

## Case Representation

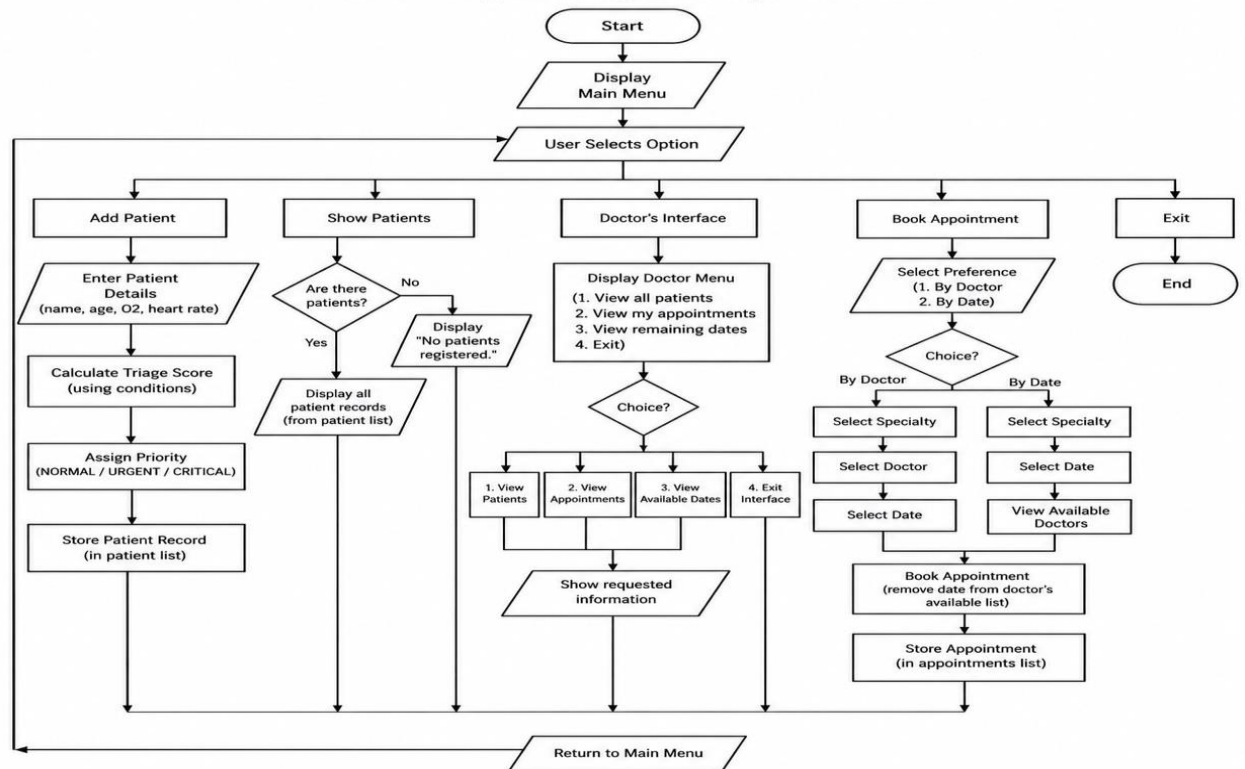
Hospital Triage System

- Add Patient
- View Patients
- Doctor Interface
- View Appointments
- View Availability
- Book Appointment
- Exit System



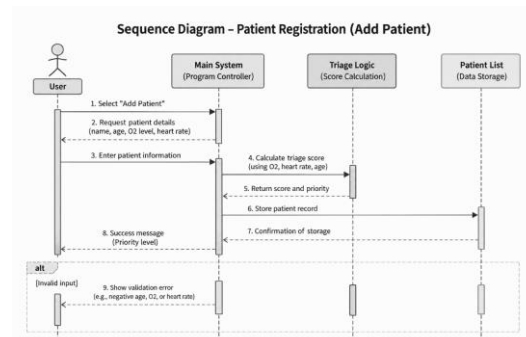
The overall workflow begins when the program starts and displays the main menu.

**Workflow Diagram – Hospital Triage Simulation**



## Sequence Diagram

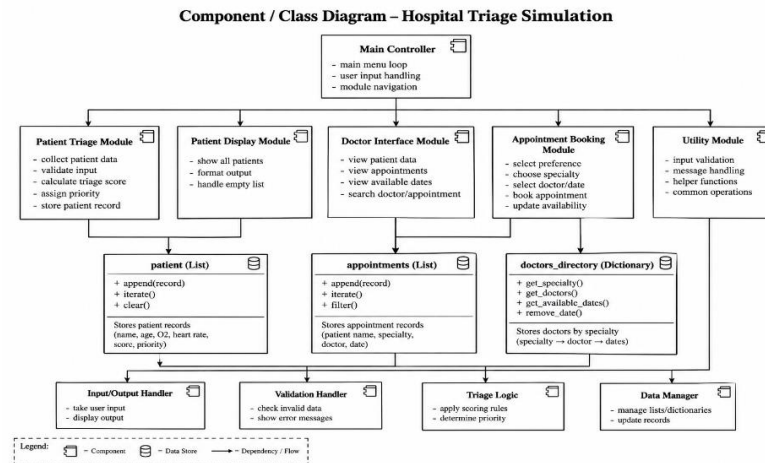
The following sequence describes a typical patient-registration operation.



## Component/Class Diagram

The current project is implemented primarily through procedural Python programming rather than separate classes.

Therefore, a component-level representation is more appropriate for the current implementation.

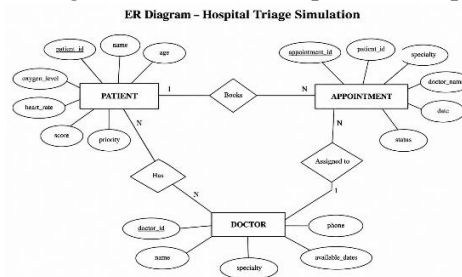


The current code does not define custom Python classes, so a traditional UML class diagram is not directly applicable to the present implementation.

## ER Diagram

The current version of the project does not use a database or persistent storage. Therefore, a traditional Entity Relationship Diagram is not required by the implementation.

However, the logical data relationships can be represented as:



This represents the logical relationship between patients, appointments, and doctors even though the data is currently stored using Python lists and dictionaries.

## Design Decisions & Rationale

### Choice of Python

Python was selected because it is the programming language used for the project and provides simple built-in structures such as lists and dictionaries.

These structures are sufficient for implementing the current simulation without requiring an external database.

### Menu-Driven Design

A menu-driven interface was selected to make the program simple to operate.

The user does not need to remember commands. Instead, the program continuously displays available operations and asks the user to select one.

This also makes the workflow easy to demonstrate during project evaluation.

### **List for Patient Records**

A list was used to store patient records because the number of patients can change during program execution.

Whenever a new patient is registered, a record is appended to the list.

This makes the implementation straightforward and appropriate for the current scope of the project.

### **Dictionary for Doctor Directory**

A nested dictionary was selected to organize doctors by specialty.

For example:

Specialty



Doctor



Available Dates

This structure allows the program to retrieve doctors based on their specialty and check their available dates.

### **Rule-Based Triage System**

The project uses a rule-based scoring mechanism instead of a machine-learning model.

This decision keeps the project aligned with the current programming scope while still demonstrating decision-making, conditional statements, data processing, and classification.

The triage score is calculated using oxygen level, heartbeat rate, and age. The resulting score determines whether the patient is categorized as NORMAL, URGENT, or CRITICAL.

The triage classification in this project is a programming simulation and should not be interpreted as a real medical diagnosis.

### **In-Memory Storage**

The current implementation stores information only while the program is running.

This approach was chosen because the current project does not require a database and focuses on demonstrating programming concepts.

As a result, data is not permanently stored after the program terminates.

### **Implementation Details**

#### **Program Initialization**

At the beginning of the program, the student's name and registration number are displayed.

Two empty lists are initialized:

```
patient = []
```

```
appointments = []
```

The program also creates a list of hospital dates and a doctor directory containing doctors, specialties, and availability.

### **Main Program Loop**

The main application operates using a continuous while True loop.

This allows the user to perform multiple operations during a single execution.

The main menu provides five choices:

1. Add patients
2. Show the patients
3. Doctor's interface
4. Book Appointment
5. Exit

The selected option determines which section of the program executes.

### **Patient Data Collection**

When the user selects the patient-registration option, the program requests:

Patient's name

Patient's age

O2 level

Heartbeat rate

The program performs basic checks for negative age, oxygen values, and heartbeat values.

---

### **Triage Score Calculation**

The program initializes:

```
score = 0
```

It then evaluates the patient's oxygen level.

```
if oxy < 90:
```

```
score = score + 5
```

```
elif oxy < 94:
```

```
score = score + 3
```

Heartbeat rate is then evaluated:

```
if heartbeat_rate > 130 or heartbeat_rate < 40:
```

score = score + 4

elif heartbeat\_rate > 110 or heartbeat\_rate < 50:

score = score + 2

Finally, one additional point is added if the patient's age is above 60.

The total score is then used to determine the patient's priority.

### **Patient Record Storage**

After processing the patient's information, the program creates a record containing the collected and calculated information.

The record includes:

- Name
- Age
- Oxygen level
- Heart rate
- Score
- Priority

The record is then added to the patient list using:

patient.append(record)

This allows the information to be accessed later through the patient-display and doctor's-interface options.

### **Appointment Booking**

The appointment system starts by asking for the patient's name.

The user can then choose between:

1. Priority by Doctor
2. Priority by Date

When booking by doctor, the user selects:

1. Specialty
2. Doctor
3. Available date

When booking by date, the user selects:

1. Specialty
2. Hospital date
3. Available doctor

After successful booking, the selected date is removed from the doctor's available dates and an appointment record is added to the appointment list.

## Screenshots / Results

This section should contain screenshots captured while running the Python program.

The recommended screenshots are:

### Screenshot 1 — Program Start

```
Name - GAURAV RATHI
Registration Number - 26BCE11407
=====VitYarthi PROJECT=====
=====Welcome to Hospital triage Simulation!=====

Which task do you want to proceed with?
```

#### Caption:

*Starting screen of the VitYarthi Hospital Triage Simulation.*

### Screenshot 2 — Main Menu

```
Which task do you want to proceed with?
1. Add patients
2. Show the patients
3. Doctor's interface
4. Book Appointment
5. Exit
Enter your preferred task : |
```

#### Caption:

*Main menu displaying the available system operations.*

### Screenshot 3 — Adding a Patient

```
Enter your preferred task : 1

Adding patient details...
Enter patient's name : Gaurav
Enter patient's age : 30
Enter patient's O2(oxygen) levels : 91
Enter heart beat value : 125
Patient added in system successfully!
Priority - URGENT!
```

#### Caption:

*Entering patient information into the system.*

### Screenshot 4 — Triage Result

```
Enter your preferred task : 2

Loading patient's details...

=== Registered Triage Patients ===
[('Name : ', 'Gaurav'), ('Patient's age', 30), ('Oxygen levels : ', 91), ('Heart rate : ', 125), ('Score : ', 5), ('Priority : ', 'URGENT!')]

```

**Caption:** *Triage score and priority generated for a registered patient.*

### Screenshot 5 — Registered Patients

```
Enter your preferred task : 2

Loading patient's details...

=== Registered Triage Patients ===
[('Name : ', 'Gaurav'), ('Patient's age', 30), ('Oxygen levels : ', 91), ('Heart rate : ', 125), ('Score : ', 5), ('Priority : ', 'URGENT!')]
[('Name : ', 'Rohan'), ('Patient's age', 32), ('Oxygen levels : ', 93), ('Heart rate : ', 130), ('Score : ', 5), ('Priority : ', 'URGENT!')]
```

#### Caption:

*Display of registered patient records.*

### Screenshot 6 — Doctor's Interface

```
Enter your preferred task : 3

--- Doctor's Interface ---
1.) View all triage patients data
2.) View my booked appointments
3.) View my remaining available dates
4.) Exit Interface
```

#### Caption:

*Doctor's interface and available operations.*

```

Which task do you want to proceed with?
1. Add patients
2. Show the patients
3. Doctor's interface
4. Book Appointment
5. Exit
Enter your preferred task : 4

--- Book an Appointment ---
Enter patient's name: Arjun

How would you like to book?
1. Priority by Doctor
2. Priority by Date
Enter your choice (1 or 2): 2

Select Specialty:
1. Cardiologist
2. Orthopedic doctor
3. Oncologist
4. Pulmonologist
Enter specialty choice (1-4): 1

Hospital Available Dates:
1. 2026-10-01
2. 2026-10-02
3. 2026-10-03
4. 2026-10-04
Select a date (1-4): 3

Cardiologist doctors available on 2026-10-03:
1. Dr. Sharma
2. Dr. Rao
Select a doctor (1-2): 1

Appointment booked successfully!
Details -> Patient: Arjun | Doctor: Dr. Sharma (Cardiologist) | Date: 2026-10-03

Which task do you want to proceed with?
1. Add patients
2. Show the patients
3. Doctor's interface
4. Book Appointment
5. Exit
Enter your preferred task : 3

--- Doctor's Interface ---
1.) View all triage patients data
2.) View my booked appointments
3.) View my remaining available dates
4.) Exit Interface
Enter your preferred task : 3
Enter Doctor's Name (e.g., Dr. Sharma): Dr. Sharma

Remaining available dates for Dr. Sharma:
- 2026-10-01
- 2026-10-02
- 2026-10-04

```

**Caption: Doctor availability**  
*Display of remaining available dates for a doctor.*

## Screenshot 8 — Appointment Booking

```
Which task do you want to proceed with?
1. Add patients
2. Show the patients
3. Doctor's interface
4. Book Appointment
5. Exit
Enter your preferred task : 4

--- Book an Appointment ---
Enter patient's name: Arjun

How would you like to book?
1. Priority by Doctor
2. Priority by Date
Enter your choice (1 or 2): 2

Select Specialty:
1. Cardiologist
2. Orthopedic doctor
3. Oncologist
4. Pulmonologist
Enter specialty choice (1-4): 1

Hospital Available Dates:
1. 2026-10-01
2. 2026-10-02
3. 2026-10-03
4. 2026-10-04
Select a date (1-4): 3

Cardiologist doctors available on 2026-10-03:
1. Dr. Sharma
2. Dr. Rao
Select a doctor (1-2): 1

Appointment booked successfully!
Details -> Patient: Arjun | Doctor: Dr. Sharma (Cardiologist) | Date: 2026-10-03
```

### Caption:

*Selecting specialty, doctor, and appointment date.*

## Screenshot 9 — Successful Appointment

```
Appointment booked successfully!
Details -> Patient: Arjun | Doctor: Dr. Sharma (Cardiologist) | Date: 2026-10-03
```

**Caption::** *Successful appointment booking confirmation.*

## Testing Approach

Testing was performed by checking the different operations of the system individually and verifying whether the expected output was produced.

### Functional Testing

| Test Case | Input/Action             | Expected Result               |
|-----------|--------------------------|-------------------------------|
| TC01      | Start program            | Main menu displayed           |
| TC02      | Select Add Patient       | Patient information requested |
| TC03      | Enter negative age       | Validation message displayed  |
| TC04      | Enter negative oxygen    | Validation message displayed  |
| TC05      | Enter negative heartbeat | Validation message displayed  |
| TC06      | Enter valid patient data | Triage score calculated       |
| TC07      | Score $\geq 6$           | CRITICAL priority             |
| TC08      | Score 3–5                | URGENT priority               |

| Test Case | Input/Action                     | Expected Result                 |
|-----------|----------------------------------|---------------------------------|
| TC09      | Score < 3                        | NORMAL priority                 |
| TC10      | Show patients with no records    | No patients message             |
| TC11      | Show patients after registration | Patient records displayed       |
| TC12      | Open Doctor's Interface          | Doctor menu displayed           |
| TC13      | Search for doctor appointment    | Matching appointments displayed |
| TC14      | Check doctor availability        | Remaining dates displayed       |
| TC15      | Book appointment by doctor       | Appointment created             |
| TC16      | Book appointment by date         | Appointment created             |
| TC17      | Invalid doctor selection         | Error message displayed         |
| TC18      | Invalid date selection           | Error message displayed         |
| TC19      | Select Exit                      | Program terminates              |

### **Triage Testing**

Different combinations of oxygen level, heartbeat rate, and age should be tested to verify the scoring logic.

For example:

#### **Test Case A**

Oxygen = 98

Heartbeat = 75

Age = 25

Expected result:

Score = 0

Priority = NORMAL

#### **Test Case B**

Oxygen = 92

Heartbeat = 115

Age = 40

Expected result:

Score = 5

Priority = URGENT

#### **Test Case C**

Oxygen = 88

Heartbeat = 140

Age = 70

Expected result:

Score = 10

Priority = CRITICAL

These test cases verify that the conditional scoring system responds appropriately to different input combinations.

## **Appointment Testing**

Appointment testing verifies that:

- Valid specialties can be selected.
  - Valid doctors can be selected.
  - Available dates can be selected.
  - Invalid selections are rejected.
- Successfully booked dates are removed from the doctor's available-date list.
  - Appointment information is stored in the appointment list.

## **Challenges Faced**

Several challenges were involved in developing the project.

### **Designing the Triage Logic**

One of the main challenges was converting patient parameters into a simple scoring mechanism.

The system needed to consider multiple inputs rather than making a decision based on only one value.

### **Managing Multiple Data Structures**

The project uses lists and nested dictionaries simultaneously.

Managing patient records, appointment records, doctors, specialties, and availability required careful organization of the data.

### **Appointment Availability**

Another challenge was managing available dates.

Once a date is booked, it must be removed from the doctor's available-date list so that the same slot is not shown as available again.

The program therefore modifies the corresponding date list after a successful booking.

## **Handling User Input**

The system depends heavily on user input.

Therefore, invalid menu selections, invalid doctor selections, invalid dates, and invalid numerical values need to be considered.

Basic validation has been included in the current implementation.

## **Maintaining the Program Flow**

Because the system contains both a main loop and a separate Doctor's Interface loop, maintaining the correct movement between menus was an important implementation challenge.

The Doctor's Interface contains its own exit option that returns control to the main menu.

## **Learnings & Key Takeaways**

The development of this project provided practical experience in applying Python programming concepts to a real-world-inspired problem.

### **Application of Conditional Statements**

The project demonstrated how if, elif, and else statements can be used to make decisions based on patient information and user selections.

### **Use of Lists**

Lists were used to dynamically store:

- Patient records
- Appointment records
- Hospital dates

This provided practical understanding of storing and modifying collections of data.

### **Use of Dictionaries**

The doctor directory demonstrated how dictionaries can be used to organize structured information.

Nested dictionaries were used to connect:

**Specialty → Doctor → Available Dates**

### **Loops**

The project extensively uses loops.

The main while True loop allows continuous interaction with the system, while for loops are used to iterate through patient records, doctors, dates, and appointments.

### **Input Validation**

The project demonstrated the importance of checking user input before using it in further processing.

Examples include checking negative values and validating menu, doctor, and date selections.

## **Problem-Solving**

The project helped translate a real-world-inspired hospital workflow into programming logic.

Instead of directly writing code, the problem was divided into smaller components such as patient registration, triage, patient display, doctor interface, and appointment booking.

## **Understanding Program State**

The appointment module provided practical experience in maintaining the state of the system.

For example, when an appointment is booked, the selected date is removed from the available-date list.

This demonstrates how a program can update stored information based on user actions.

## **Future Enhancements**

The current version provides the core functionality of a hospital triage and appointment simulation. Several enhancements could be implemented in future versions.

### **Database Integration**

A database such as SQLite or MySQL could be introduced to permanently store:

- Patient records
- Doctor information
- Appointments
- Availability

This would prevent data from being lost when the program terminates.

---

## **Graphical User Interface**

The current application uses a command-line interface.

A future version could provide a graphical user interface using an appropriate Python GUI framework.

This could make the system easier for users who are not comfortable with command-line applications.

### **Separate User Roles**

Future versions could introduce separate interfaces for:

- Patients
- Doctors
- Hospital administrators

This would allow different users to access only the operations relevant to them.

### **Patient Search**

A search feature could be introduced to find a patient using information such as:

- Patient name
- Patient ID
- Age

This would become increasingly useful as the number of records grows.

### **Appointment Cancellation and Rescheduling**

The current system supports appointment booking.

Future versions could allow users to:

- Cancel appointments
- Reschedule appointments
- View appointment history

### **More Detailed Triage Parameters**

The current triage simulation uses oxygen level, heartbeat rate, and age.

Future versions could incorporate additional parameters such as other patient-provided observations, provided that the scoring rules are clearly defined and treated as a simulation rather than clinical diagnosis.

### **Persistent Reports**

The system could generate a report containing:

- Number of patients
- Number of critical cases
- Number of urgent cases
- Number of normal cases
- Number of appointments
- Doctor-wise appointments

This could help demonstrate the reporting and analytics aspect of the project.

## **References**

The following sources were used during the development and documentation of the project:

1. **VITyarthi – Build Your Own Project: General Project Instructions & Submission Guidelines**, provided as part of the course project requirements. The document specifies the required report structure, design artefacts, GitHub requirements, and evaluation criteria.
2. **Python Project Source Code – VITyarthi Hospital Triage Simulation**, containing the implementation of patient registration, triage scoring, doctor interface, doctor availability, and appointment booking.
3. **Python Programming Language Documentation** — used as a general reference for Python syntax and built-in programming constructs used in the implementation.

## **Conclusion**

The VITYarthi Hospital Triage Simulation demonstrates the application of fundamental programming concepts to a practical hospital-management scenario. The system combines patient data collection, rule-based triage classification, doctor availability management, and appointment booking within a single menu-driven application.

The project provides an implementation-based demonstration of conditional statements, loops, lists, dictionaries, input handling, validation, and data processing. The current implementation establishes the core functionality while leaving several opportunities for future expansion, including database integration, graphical interfaces, role-based access, persistent records, and enhanced reporting.

The project therefore provides a foundation that can be extended into a more comprehensive hospital-management simulation while maintaining the core concepts implemented in the current Python version.